

Assignment 2

INF-2300

Erling Heimstad Willassen

1. Introduction

Clients and servers can communicate with each other by using CURL (Create, Read, Update, Delete) commands through HTTP (HyperText Transfer Protocol). This way of communication can give a client the layout of a web-page in HTML (HyperText Markup Language) and Javascript. In this assignment, the student will create a web-page in HTML which will communicate with a provided server in JavaScript that will show and be able to manipulate a list on the server side.

2. Technical Background

HTML is a very much used markup language to display web pages on the internet. HTML is a document file that contains code describing the layout of the page content and how it will be display by a client browser [1]. By entering a DNS web-page domain or IP address in a client browser [2], the client browser will send a HTTP request to said server to fetch a HTML layout, file(s), or an arbitrary object from the server. The server side handles these request specifically, allowing or disallowing certain addresses to be accessed and return information to the client. This way, the server dictates what the client is allowed to fetch and to be displayed in the browser through HTTP requests. In addition to this, these HTTP requests can also be used to manipulate the state of internal objects or files on the server side if the server have methods to handle these requests.

JavaScript is a lightweight programming language that is often used on web-pages to display and run functions in a web-browser [3]. Web-browsers have built-in JavaScript engine that can run JavaScript codes in HTML layouts sent from a server [3]. Since a HTML file only provide a static web-page to be shown on the client side, JavaScript can add a more dynamic interface by running functions on certain objects in the HTML code that are interacted on from the client side. This integration between HTML objects and JavaScript functions can be used to send specific requests to a web-browser server to fetch certain information to be displayed on the client web-browser.

Flask is a web application framework designed to the coding language Python to make setting up a server quick and easy [4]. By using the Flask framework in a python code, creating a server is an easy task compared to traditional methods. The Flask framework comes with built-in methods to handle request and response from and to clients, respectively, making it a simple task to create a server in short amount of time.

3. Design

The design of the implementation to the assignment was to create a static HTML web-page to be displayed on the client side. This file would be available on the server to be accessed by clients by

sending a GET request to the homepage of the site with the domain of "/" and "/index.html", where the latter is the name of the HTML file. The HTML file would contain the layout of the site containing objects to use for different HTTP requests. The objects would be connected to JavaScript functions in the HTML file, where the different HTTP requests are handled and displayed in a fashionably order. Since the interface has to be user-friendly for the user on the client side, each object to handle HTTP requests are descriptive to inform the user on its usage and contain error-handling so bad requests are less likely to happen. In the pre-code to the assignment, a pre-written code for the server is provided with the different requests available to be used. These requests addresses will be used for the static HTML layout. These requests will be used to handle modification to a list that is based on the server side. By using these objects displayed on the client side, a user can manipulate the state of a list that is preserved on the server side of the interaction.

4. Implementation

As described in the chapter 3 for Design, a pre-written code for the server was provided with the assignment. In this python file all the requests allowed by a client to perform were already given, except for accessing the "index.html" HTML file for web-page. This was added to the server code so that when a client accesses the IP address or DNS domain for the server or with a trailing "/index.html", the server will provide the HTML layout for the static web-page. By using the `render_template()` function from the Flask framework, the file of "index.html" can be easily responded to the client.

```
@app.route("/", methods=["GET"])
def index():
    return render_template('index.html')

@app.route("/index.html", methods=["GET"])
def index_html():
    return render_template('index.html')
```

Figure 1: Code for routing a GET request to client

For the HTML file, the student had to create it from scratch. By using starting templates for a web-page layout in HTML, a client would be shown the layout of "index.html" when accessing the IP address of the server. Here, objects such as text input and buttons would handle HTTP requests to the server to alter the state of a list object called "items" that contains a dictionary/json of the TODOs on the server side.

```
items = [  
  {"id": 1, "name": "Create a client application", "done": False},  
  {"id": 2, "name": "Write an excellent report", "done": False},  
  {"id": 3, "name": "Jazz it up with some cool features?", "done": False},  
]
```

Figure 2: Item list on server side to have its state altered by HTTP requests.

One obstacle was how to connect these objects that a user can use on the client side to alter the state of a list on the server side. First, was to find out how to connect one HTML object to a JavaScript function in the same HTML file. This was solved by creating a "<form></form>" container object to store user input objects. Here, the container will be waiting for a "onsubmit=" event that registers when a user submits a query and/or ID on the objects inside it. By putting a JavaScript function name at the end of the event, the HTML file would initialize that specific function. From here, the student had to figure out how to handle specific HTTP requests from these containers to the server.

By using JavaScript, the student managed to create functions that could handle requests to the server by using an embedded JavaScript methods called `fetch()`. This method handles the sending of requests to the server with a string containing the path to elements and what CRUD method to use. This way, the handling of each CRUD request that the server can respond to can be handled specifically in JavaScript functions. Therefore, the student created specific functions to handle different HTTP requests to the internal "items" list on the server side, depending on what HTML objects were manipulated on the client side. By doing this, the user on the client side can manipulate the list after their wish when interacting with HTML objects that handles specific HTTP requests.

To have a more dynamic web-page with little delay, the student created functions with the built-in methods of "async function" in JavaScript. When a function is initialized, it will happen synchronously and all other running functions would stop until that function is finalized. With using the "async" keyword, the JavaScript code will be told that it can perform other functions at the same time. This is important if there is a delay in a response from a server that can take some time for a object to be received. If there is a delay and the function has to wait for the received object, all other functions will freeze until the waiting function can be resolved. This could give the client a freezing experience when handling with HTML objects that could take time to finalize. With using "async", this can be avoided and the user can handle other HTML objects at the same time without waiting for each object to be resolved. Combined with the "await" keyword methods such as "`fetch()`", the JavaScript engine is told to wait for the input from servers and not to keep going without getting a respond from server first. This avoids that functions continue with handling of objects without getting a server reply first and getting errors on the client side. To have a more dynamic webpage, the method of "`e.preventDefault()`" was also used. This method prevents the page to reload automatically by the web-browser, but lets the JavaScript code to handle the update of the webpage without reloading or opening a new tab.

```

async function submitID(e){ // Get Query / Submit ID GET function
  e.preventDefault();
  var numberID = document.getElementById("inputID").value.trim();
  const response = await fetch(`/api/items/${encodeURIComponent(numberID)}`);
  const object = await response.json();
  const status = document.getElementById('getStatus');
  if (!object.item){
    status.innerHTML = object.Description;
    return false
  }
  status.innerHTML = ""; // resetting text in case of earlier error code
  createTable(object)
}

```

Figure 3: An example of how a JavaScript function looks like in the HTML file.

5. Results

When accessing the IP address of the server in a client web-browser, the HTML layout code of "index.html" will be shown in the web browser as intended as seen below.

TODO List

This site will handle your TODO list. You will get six options to use on your TODO list

1. **Submit Query:**
Submits a new TODO query to the list of TODOs
2. **Get Query:**
Submit the ID number of a TODO query to see it individually
3. **Change Query:**
Change an existing TODO with ID number
4. **Delete Query:**
Submit the ID number of query to be deleted
5. **Click to fetch list:**
Show all TODOs in the TODO list
6. **Click to remove list:**
Will remove all TODOs in the list

Submit Query:

Submit query text to insert into TODO list.

Get Query:

Submit query ID to fetch from list.

Change Query:

Submit query ID to change from list.

Delete Query:

Submit query ID to delete from list.

TODO list

ID	Name	Done
1	Create a client application	☐
2	Write an excellent report	☐
3	Jazz it up with some cool features?	☐

Figure 4: How the HTML layout would look on a web-client.

Here, the user on the client side can perform seven actions to the HTML layout. Each actions have their own objects that are connected to specific JavaScript functions to initialize. These actions will perform in chronological order:

1. Submit Query: POST request to list of query
2. Get Query: GET request to get specific ID of a query to be displayed
3. Change Query: PUT request to change the query to a specific ID of a query
4. Delete Query: DELETE request to delete an item in the list from a specific ID of a query
5. Click to fetch list: GET request to acquire the whole list from the server
6. Click to remove list: DELETE request on all items in list

These options are connected to specific HTTP requests to manipulate the state of the list on server side, and works as intended.

6. Discussion

The HTML file was used a static web-page to display and perform actions against the item list on the server side. This seemed to the student as the easiest way to display and manipulate the list stored on the server. Changing HTML objects depending on the interaction from the user on the client side was chosen as the most straight-forward way to alter the state of the server list and shown in the web-browser. The main reason for using a static website is because the student was not much familiar with HTML or JavaScript development and by creating a more dynamic site would deem to difficult and hard to implement. By changing the HTML object on the client side from the inputs of a user was shown to be a more beginner friendly way to create a website with HTML.

Finding a method to use JavaScript function to handle requests to server was a time demanding task for the student. Even when managing to send a request through a JavaScript function in a HTML site, often errors would occur on the client site even if the server had acquired the request and handled it correctly. After some research, it was deemed that the problem was that functions did not wait for the response from the server to continue further down in the function. The function continued on with handling of objects that were non-existing before the server had sent data back to the function. This is where async functions and await keywords had to be used for functions to tell them to wait for the server response before continuing on. This is was a new element for the student that was used to functions acting quickly and autonomous without implicating that the object had to be first acquired before continuing. When the student figured out this trick, the rest of the implementation went more smoothly and a functional website from a HTML file could be created more easily.

Even if the student this not alter the python code for the server much, using the Flask framework seemed to be a very user-friendly way to create a server. This shows that creating a server does not need to be a difficult task and can be easily implemented without much coding to get one up and running.

7. Conclusion

The student was tasked to implement a web client with a user-friendly interface to handle HTTP requests on a list residing on the server side. The student created a static website by using a HTML file with JavaScript functionality to handle user interactions. These interactions would then send HTTP requests to server to fetch or alter the state of the internal server list of items. The student managed to create a website that was user-friendly and could display and change list of items based on the server.

8. Sources

- [1] Wikipedia Contributors, "HTML," *Wikipedia*, Nov. 27, 2018.
<https://en.wikipedia.org/wiki/HTML>
- [2] Wikipedia Contributors, "Domain Name System," *Wikipedia*, Aug. 09, 2019.
https://en.wikipedia.org/wiki/Domain_Name_System
- [3] Wikipedia Contributors, "JavaScript," *Wikipedia*, Mar. 22, 2019.
<https://en.wikipedia.org/wiki/JavaScript>
- [4] Flask, "Welcome to Flask — Flask Documentation (3.0.x)," *Palletsprojects.com*, 2010.
<https://flask.palletsprojects.com/en/stable/>